

1. Předpoklady a validační schéma

Všechny příkazy jsou psány pro spuštění v mongosh nad databází yelp. Datový model vychází z Yelp Open Datasetu a pracuje se třemi hlavními kolekcemi: businesses, reviews a users. Dotazy jsou seřazeny tak, aby na sebe navazovaly. Pomocné analytické kolekce jsou vytvářeny pouze pro účely demonstrace práce s daty a následných ověřovacích dotazů.

1.1 Doporučená ukázka validačního schématu

MongoDB část zadání vyžaduje validační schéma. Následující ukázka nastavuje základní JSON Schema validátory pro tři hlavní kolekce. V ukázce je použito validationAction: "error".

```
db.runCommand({
  collMod: "businesses",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["business_id", "name", "city", "state", "stars"],
      properties: {
        business_id: { bsonType: "string" },
        name: { bsonType: "string" },
        city: { bsonType: "string" },
        state: { bsonType: "string" },
        stars: { bsonType: ["double", "int"] },
        review_count: { bsonType: ["int", "long", "double"] },
        is_open: { bsonType: ["int", "bool"] }
      }
    }
  },
  validationLevel: "moderate",
  validationAction: "error"
});
```

```
db.runCommand({
  collMod: "reviews",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["review_id", "user_id", "business_id", "stars"],
      properties: {
        review_id: { bsonType: "string" },
        user_id: { bsonType: "string" },
        business_id: { bsonType: "string" },
        stars: { bsonType: ["double", "int"] },
        useful: { bsonType: ["int", "long", "double"] },
        funny: { bsonType: ["int", "long", "double"] },
        cool: { bsonType: ["int", "long", "double"] }
      }
    }
  },
  validationLevel: "moderate",
```

```

    validationAction: "error"
  });

  db.runCommand({
    collMod: "users",
    validator: {
      $jsonSchema: {
        bsonType: "object",
        required: ["user_id", "name"],
        properties: {
          user_id: { bsonType: "string" },
          name: { bsonType: "string" },
          review_count: { bsonType: ["int", "long", "double"] },
          useful: { bsonType: ["int", "long", "double"] },
          funny: { bsonType: ["int", "long", "double"] },
          cool: { bsonType: ["int", "long", "double"] }
        }
      }
    },
    validationLevel: "moderate",
    validationAction: "error"
  });

```

2. Kategorie 1 - Práce s daty

Dotaz 1: Vložení záznamu o analytickém běhu do pomocné kolekce

Úloha: vložení záznamu o analytickém běhu do pomocné kolekce

```

const runDoc = {
  run_type: "initial_analysis",
  createdAt: new Date(),
  business_count: db.businesses.countDocuments(),
  review_count: db.reviews.countDocuments(),
  user_count: db.users.countDocuments()
};

db.analysis_run_log.insertOne(runDoc);

db.analysis_run_log.find(
  {},
  { _id: 0, run_type: 1, createdAt: 1, business_count: 1, review_count: 1, user_count: 1 }
).sort({ createdAt: -1 }).limit(1);

```

Vysvětlení: Příkaz insertOne vloží jeden dokument do pomocné kolekce analysis_run_log. Dokument obsahuje typ běhu, čas vytvoření a aktuální počty dokumentů ve třech hlavních kolekcích. Ověřovací find zobrazí poslední vložený běh a vrátí konkrétní data z databáze.

Dotaz 2: Vytvoření souhrnu recenzí podle podniků pomocí \$merge

Úloha: vytvoření souhrnu recenzí podle podniků pomocí \$merge

```
db.reviews.aggregate([
  {
    $group: {
      _id: "$business_id",
      review_count: { $sum: 1 },
      avg_review_stars: { $avg: "$stars" },
      total_useful: { $sum: { $ifNull: ["$useful", 0] } }
    }
  },
  {
    $lookup: {
      from: "businesses",
      localField: "_id",
      foreignField: "business_id",
      as: "business"
    }
  },
  { $unwind: "$business" },
  {
    $project: {
      _id: "$_id",
      business_id: "$_id",
      business_name: "$business.name",
      city: "$business.city",
      state: "$business.state",
      stored_stars: "$business.stars",
      review_count: 1,
      avg_review_stars: { $round: ["$avg_review_stars", 2] },
      total_useful: 1
    }
  },
  { $sort: { review_count: -1 } },
  {
    $merge: {
      into: "analysis_business_review_summary",
      whenMatched: "replace",
      whenNotMatched: "insert"
    }
  }
], { allowDiskUse: true });

db.analysis_business_review_summary.find(
  {},
  { _id: 0, business_id: 1, business_name: 1, city: 1, state: 1, review_count: 1, avg_review_stars: 1,
    total_useful: 1 }
).sort({ review_count: -1 }).limit(20);
```

Vysvětlení: Agregace seskupí recenze podle business_id, vypočítá počet recenzí, průměrné hodnocení a součet useful reakcí. Pomocí \$lookup se připojí informace o podniku z kolekce businesses. Výsledek je uložen do pomocné kolekce analysis_business_review_summary přes \$merge. Jako _id je ponecháno business_id, takže opakované spuštění dotazu nahrazuje existující souhrny místo vytváření duplicit.

Dotaz 3: Vytvoření souhrnu uživatelů podle roku registrace

Úloha: vytvoření souhrnu uživatelů podle roku registrace

```
db.users.aggregate([
  {
    $project: {
      yelping_year: {
        $substrBytes: [
          { $ifNull: [{ $toString: "$yelping_since" }, "unknown"] },
          0,
          4
        ]
      },
      review_count: { $ifNull: ["$review_count", 0] },
      useful: { $ifNull: ["$useful", 0] },
      funny: { $ifNull: ["$funny", 0] },
      cool: { $ifNull: ["$cool", 0] }
    }
  },
  {
    $group: {
      _id: "$yelping_year",
      user_count: { $sum: 1 },
      avg_review_count: { $avg: "$review_count" },
      avg_useful: { $avg: "$useful" },
      avg_funny: { $avg: "$funny" },
      avg_cool: { $avg: "$cool" }
    }
  },
  {
    $project: {
      _id: "$_id",
      yelping_year: "$_id",
      user_count: 1,
      avg_review_count: { $round: ["$avg_review_count", 2] },
      avg_useful: { $round: ["$avg_useful", 2] },
      avg_funny: { $round: ["$avg_funny", 2] },
      avg_cool: { $round: ["$avg_cool", 2] }
    }
  },
  { $sort: { yelping_year: 1 } },
  {
    $merge: {
      into: "analysis_user_year_summary",
      whenMatched: "replace",
      whenNotMatched: "insert"
    }
  }
])
```

```

    }
  ], { allowDiskUse: true });

db.analysis_user_year_summary.find(
  {},
  { _id: 0, yelping_year: 1, user_count: 1, avg_review_count: 1, avg_useful: 1, avg_funny: 1, avg_cool: 1 }
).sort({ yelping_year: 1 });

```

Vysvětlení: Pipeline z pole `yelping_since` odvodí rok registrace uživatele, seskupí uživatele podle roku a vypočítá průměrnou aktivitu. Výsledek je uložen do `analysis_user_year_summary`. Dotaz je navazující, protože vytváří další analytickou kolekci, se kterou lze následně pracovat.

Dotaz 4: Vytvoření souhrnu podniků podle státu

Úloha: vytvoření souhrnu podniků podle státu

```

db.businesses.aggregate([
  {
    $match: {
      state: { $exists: true, $nin: [null, ""] }
    }
  },
  {
    $group: {
      _id: "$state",
      business_count: { $sum: 1 },
      open_business_count: {
        $sum: {
          $cond: [{ $eq: ["$is_open", 1] }, 1, 0]
        }
      },
      avg_stars: { $avg: "$stars" },
      avg_review_count: { $avg: "$review_count" }
    }
  },
  {
    $project: {
      _id: "$_id",
      state: "$_id",
      business_count: 1,
      open_business_count: 1,
      avg_stars: { $round: ["$avg_stars", 2] },
      avg_review_count: { $round: ["$avg_review_count", 2] }
    }
  },
  { $sort: { business_count: -1 } },
  {
    $merge: {
      into: "analysis_state_business_summary",
      whenMatched: "replace",
      whenNotMatched: "insert"
    }
  }
]

```

```

    }
  ], { allowDiskUse: true });

db.analysis_state_business_summary.find(
  {},
  { _id: 0, state: 1, business_count: 1, open_business_count: 1, avg_stars: 1, avg_review_count: 1 }
).sort({ business_count: -1 }).limit(20);

```

Vysvětlení: Dotaz vytváří souhrnnou kolekci podniků podle státu. Opravený filtr používá \$nin: [null, ""], protože dva operátory \$ne nad stejným polem by v jednom JavaScript objektu nebyly bezpečné. Výsledek vrátí počty podniků, počty otevřených podniků a průměrné hodnoty.

Dotaz 5: Doplnění příznaku high_volume pomocí pipeline update

Úloha: doplnění příznaku high_volume pomocí pipeline update

```

db.analysis_business_review_summary.updateMany(
  {},
  [
    {
      $set: {
        high_volume: { $gte: ["$review_count", 50] }
      }
    }
  ]
);

db.analysis_business_review_summary.find(
  { high_volume: true },
  { _id: 0, business_id: 1, business_name: 1, review_count: 1, high_volume: 1 }
).sort({ review_count: -1 }).limit(20);

```

Vysvětlení: Příkaz updateMany je zde použit ve formě update pipeline. Hodnota high_volume není nastavena staticky, ale je vypočtena z hodnoty review_count. Podniky s alespoň 50 recenzemi jsou označeny jako vysokokapacitní z hlediska objemu recenzí.

Dotaz 6: Ukázka mazání dat nad pomocnou kolekcí

Úloha: ukázka mazání dat nad pomocnou kolekcí

```

db.analysis_delete_demo.drop();

db.analysis_business_review_summary.aggregate([
  { $sort: { review_count: 1 } },
  { $limit: 20 },
  {
    $setWindowFields: {
      sortBy: { review_count: 1 },
      output: {
        row_number: { $documentNumber: {} }
      }
    }
  }
]);

```

```

    },
    {
      $set: {
        delete_candidate: { $lte: ["$row_number", 5] }
      }
    },
    { $unset: "row_number" },
    {
      $merge: {
        into: "analysis_delete_demo",
        whenMatched: "replace",
        whenNotMatched: "insert"
      }
    }
  ], { allowDiskUse: true });

db.analysis_delete_demo.find(
  {},
  { _id: 0, business_id: 1, business_name: 1, review_count: 1, delete_candidate: 1 }
).sort({ delete_candidate: -1, review_count: 1 }).limit(10);

db.analysis_delete_demo.deleteMany({ delete_candidate: true });

db.analysis_delete_demo.find(
  {},
  { _id: 0, business_id: 1, business_name: 1, review_count: 1, delete_candidate: 1 }
).sort({ review_count: 1 }).limit(10);

```

Vysvětlení: Tento dotaz bezpečně demonstruje deleteMany nad pomocnou kolekcí, nikoliv nad hlavními importovanými daty. Nejprve se vytvoří testovací kolekce analysis_delete_demo z již vytvořeného souhrnu podniků. Prvních pět dokumentů je označeno jako delete_candidate, následně jsou smazány a poslední find vrací zbývající dokumenty. Tím je splněno, že dotaz po mazání stále vrací data.

3. Kategorie 2 - Agregční funkce

Dotaz 7: Výpočet nejlépe hodnocených kategorií podniků

Úloha: výpočet nejlépe hodnocených kategorií podniků

```

db.businesses.aggregate([
  {
    $project: {
      category_list: {
        $cond: [
          { $isArray: "$categories" },
          "$categories",
          {
            $cond: [
              { $or: [{ $eq: ["$categories", null] }, { $eq: ["$categories", ""] } ] },
              [],

```

```

        { $split: ["$categories", ","] }
    ]
}
],
stars: 1
}
},
{ $unwind: "$category_list",
{
    $set: {
        category: { $trim: { input: "$category_list" } }
    }
},
{ $match: { category: { $nin: [null, ""] } } },
{
    $group: {
        _id: "$category",
        avg_stars: { $avg: "$stars" },
        business_count: { $sum: 1 }
    }
},
{ $match: { business_count: { $gte: 30 } } },
{
    $project: {
        _id: 0,
        category: "$_id",
        avg_stars: { $round: ["$avg_stars", 2] },
        business_count: 1
    }
},
{ $sort: { avg_stars: -1, business_count: -1 } },
{ $limit: 20 }
], { allowDiskUse: true });

```

Vysvětlení: Dotaz pracuje s kategoriemi podniků. Je ošetřeno, zda je pole categories již polem, nebo původním textovým řetězcem odděleným čárkami. Po rozbalení kategorií se počítá průměrné hodnocení a počet podniků v každé kategorii. Do výsledku vstupují jen kategorie s dostatečným počtem podniků.

Dotaz 8: Porovnání států podle průměrného hodnocení podniků

Úloha: porovnání států podle průměrného hodnocení podniků

```

db.businesses.aggregate([
{
    $match: {
        state: { $exists: true, $nin: [null, ""] }
    }
},
{
    $group: {
        _id: "$state",
        business_count: { $sum: 1 },

```

```

open_business_count: {
  $sum: {
    $cond: [{ $eq: ["$is_open", 1] }, 1, 0]
  }
},
avg_stars: { $avg: "$stars" },
avg_review_count: { $avg: "$review_count" }
}
},
{ $match: { business_count: { $gte: 20 } } },
{
  $project: {
    _id: 0,
    state: "$_id",
    business_count: 1,
    open_business_count: 1,
    avg_stars: { $round: ["$avg_stars", 2] },
    avg_review_count: { $round: ["$avg_review_count", 2] }
  }
},
{ $sort: { avg_stars: -1, business_count: -1 } }
], { allowDiskUse: true });

```

Vysvětlení: Agregace seskupuje podniky podle státu a porovnává jejich průměrné hodnocení. Filtr \$nin odstraňuje prázdné a neexistující hodnoty. Výsledek je jiný než v předchozím dotazu, protože se nepracuje s kategoriemi, ale s geografickým členěním podle státu.

Dotaz 9: Analýza let registrace uživatelů podle průměrné aktivity

Úloha: analýza let registrace uživatelů podle průměrné aktivity

```

db.users.aggregate([
{
  $project: {
    yelping_year: {
      $substrBytes: [
        { $ifNull: [{ $toString: "$yelping_since" }, "unknown"] },
        0,
        4
      ]
    },
    review_count: { $ifNull: ["$review_count", 0] },
    engagement_score: {
      $add: [
        { $ifNull: ["$useful", 0] },
        { $ifNull: ["$funny", 0] },
        { $ifNull: ["$cool", 0] }
      ]
    }
  }
},
{
  $sort: {
    yelping_year: -1,
    review_count: -1,
    engagement_score: -1
  }
}
], { allowDiskUse: true });

```

```

$group: {
  _id: "$yelping_year",
  user_count: { $sum: 1 },
  avg_review_count: { $avg: "$review_count" },
  avg_engagement_score: { $avg: "$engagement_score" }
},
{ $match: { user_count: { $gte: 10 } } },
{
  $project: {
    _id: 0,
    yelping_year: "$_id",
    user_count: 1,
    avg_review_count: { $round: ["$avg_review_count", 2] },
    avg_engagement_score: { $round: ["$avg_engagement_score", 2] }
  }
},
{ $sort: { avg_engagement_score: -1 } }
], { allowDiskUse: true });

```

Vysvětlení: Dotaz vytváří odvozený engagement_score jako součet užitečných, vtipných a cool reakcí. Poté seskupuje uživatele podle roku registrace a porovnává průměrnou aktivitu jednotlivých ročníků.

Dotaz 10: Města s nejvyšším průměrným počtem recenzí u otevřených podniků

Úloha: města s nejvyšším průměrným počtem recenzí u otevřených podniků

```

db.businesses.aggregate([
{
  $match: {
    is_open: 1,
    city: { $exists: true, $nin: [null, ""] }
  }
},
{
  $group: {
    _id: {
      city: "$city",
      state: "$state"
    },
    business_count: { $sum: 1 },
    avg_review_count: { $avg: "$review_count" },
    avg_stars: { $avg: "$stars" }
  }
},
{ $match: { business_count: { $gte: 10 } } },
{
  $project: {
    _id: 0,
    city: "$_id.city",
    state: "$_id.state",
    business_count: 1,

```

```

    avg_review_count: { $round: ["$avg_review_count", 2] },
    avg_stars: { $round: ["$avg_stars", 2] }
  },
  {
    $sort: {
      avg_review_count: -1,
      avg_stars: -1
    }
  },
  { $limit: 20 }
], { allowDiskUse: true });

```

Vysvětlení: Tento dotaz vyhodnocuje otevřené podniky podle města a státu. Nevrací stejné údaje jako dotaz podle státu, protože porovnává konkrétní města a zaměřuje se na průměrný počet recenzí jako ukazatel popularity.

Dotaz 11: Porovnání uloženého hodnocení podniku s průměrem z recenzí

Úloha: porovnání uloženého hodnocení podniku s průměrem z recenzí

```

db.reviews.aggregate([
  {
    $project: {
      business_id: 1,
      stars: 1
    }
  },
  {
    $group: {
      _id: "$business_id",
      real_avg_stars: { $avg: "$stars" },
      real_review_count: { $sum: 1 }
    }
  },
  { $match: { real_review_count: { $gte: 20 } } },
  {
    $lookup: {
      from: "businesses",
      localField: "_id",
      foreignField: "business_id",
      as: "business"
    }
  },
  { $unwind: "$business" },
  {
    $project: {
      _id: 0,
      business_id: "$_id",
      business_name: "$business.name",
      stored_stars: "$business.stars",
      real_avg_stars: { $round: ["$real_avg_stars", 2] },
      real_review_count: 1,

```

```

    diff: {
      $round: [
        { $subtract: ["$real_avg_stars", "$business.stars"] },
        2
      ]
    }
  },
  { $match: { diff: { $ne: 0 } } },
  { $sort: { diff: -1 } },
  { $limit: 20 }
], { allowDiskUse: true });

```

Vysvětlení: Agregace přepočítává skutečný průměr hvězdiček z recenzí a porovnává ho s hodnotou uloženou v dokumentu podniku. \$lookup propojuje kolekce reviews a businesses. Dotaz je užitečný pro kontrolu konzistence mezi denormalizovanými a detailními daty.

Dotaz 12: Uživatelé s nejdelšími recenzemi a jejich průměrné hodnocení

Úloha: uživatelé s nejdelšími recenzemi a jejich průměrné hodnocení

```

db.reviews.aggregate([
  {
    $project: {
      user_id: 1,
      stars: 1,
      text_length: { $strLenCP: { $ifNull: ["$text", ""] } }
    }
  },
  {
    $group: {
      _id: "$user_id",
      avg_text_length: { $avg: "$text_length" },
      avg_given_stars: { $avg: "$stars" },
      review_count: { $sum: 1 }
    }
  },
  { $match: { review_count: { $gte: 10 } } },
  {
    $lookup: {
      from: "users",
      localField: "_id",
      foreignField: "user_id",
      as: "user"
    }
  },
  { $unwind: "$user" },
  {
    $project: {
      _id: 0,
      user_id: "$_id",
      user_name: "$user.name",

```

```

    avg_text_length: { $round: ["$avg_text_length", 2] },
    avg_given_stars: { $round: ["$avg_given_stars", 2] },
    review_count: 1
  },
  { $sort: { avg_text_length: -1 } },
  { $limit: 10 }
], { allowDiskUse: true });

```

Vysvětlení: Dotaz pracuje s délkou textu recenze přes \$strLenCP, následně seskupuje recenze podle uživatele a připojuje uživatelská jména z kolekce users. Výsledkem jsou uživatelé, kteří píší v průměru nejdelší recenze.

4. Kategorie 3 - Konfigurační dotazy

Dotaz 13: Zobrazení stavu shardovaného clusteru

Úloha: zobrazení stavu shardovaného clusteru

```
sh.status();
```

Vysvětlení: Příkaz sh.status() vrací přehled shardovaného clusteru. Vypisuje shardované databáze, shardované kolekce, shard key, počet chunků a rozložení dat mezi shardy. Slouží jako základní důkaz, že je cluster skutečně spuštěn v režimu shardingu.

Dotaz 14: Výpis statistik všech kolekcí v databázi yelp

Úloha: výpis statistik všech kolekcí v databázi yelp

```

db.getCollectionNames().forEach(function(collection) {
  const stats = db[collection].stats();
  print("Kolekce: " + collection);
  print("Počet dokumentů: " + stats.count);
  print("Velikost kolekce: " + stats.size + " B");
  print("Indexy: " + JSON.stringify(stats.indexSizes));
  print("Sharding: " + JSON.stringify(stats.sharded));
  print("---");
});

```

Vysvětlení: Příkaz projde všechny kolekce v databázi a pro každou zobrazí počet dokumentů, velikost, velikosti indexů a informaci o shardingu. Jde o administrační výpis, který vrací jiná data než analytické dotazy nad obsahem Yelp datasetu.

Dotaz 15: Zobrazení shard klíčů a indexů kolekcí

Úloha: zobrazení shard klíčů a indexů kolekcí

```

db.getCollectionNames().forEach(function(collection) {
  const info = db.getSiblingDB("config").collections.findOne({
    _id: "yelp." + collection
  });
});

```

```

print("Kolekce: " + collection);

if (info && info.key) {
  print("Sharding klíč: " + JSON.stringify(info.key));
  print("Unikátní: " + (info.unique ? "ano" : "ne"));
} else {
  print("Sharding klíč: neshardovaná kolekce");
}

print("Indexy:");
db[collection].getIndexes().forEach(function(idx) {
  printjson(idx);
});
print("---");
});

```

Vysvětlení: Dotaz kombinuje metadata z konfigurační databáze config s výpisem indexů přímo z jednotlivých kolekcí. U shardovaných kolekcí zobrazí shard key, u ostatních uvede, že nejsou shardované. Tento výpis je vhodný jako důkaz konfigurace clusteru.

Dotaz 16: Zobrazení validačních schémat hlavních kolekcí

Úloha: zobrazení validačních schémat hlavních kolekcí

```

["businesses", "reviews", "users"].forEach(function(collectionName) {
  const info = db.getCollectionInfos({ name: collectionName })[0];

  print("Kolekce: " + collectionName);
  printjson({
    validator: info.options.validator,
    validationLevel: info.options.validationLevel,
    validationAction: info.options.validationAction
  });
  print("---");
});

```

Vysvětlení: Tento konfigurační dotaz vrací metadata kolekcí businesses, reviews a users. Pokud bylo validační schéma nastaveno, je viditelné v části options.validator. Dotaz tím prokazuje splnění požadavku, že MongoDB řešení obsahuje validační schéma.

Dotaz 17: Zobrazení rozdělení kolekce reviews mezi shardy

Úloha: zobrazení rozdělení kolekce reviews mezi shardy

```

db.reviews.getShardDistribution();

```

Vysvětlení: Příkaz getShardDistribution() zobrazí, jak je kolekce reviews rozložena mezi jednotlivé shardy. Výpis obsahuje množství dat, počet dokumentů a procentuální podíl na jednotlivých shardech. Je to praktický důkaz, že kolekce není pouze uložena na jednom místě.

Dotaz 18: Výpis chunků a jejich přiřazení ke shardům

Úloha: výpis chunků a jejich přiřazení ke shardům

use config

```
db.chunks.aggregate([
  {
    $lookup: {
      from: "collections",
      localField: "uuid",
      foreignField: "uuid",
      as: "collectionInfo"
    }
  },
  { $unwind: "$collectionInfo" },
  {
    $match: {
      "collectionInfo.id": /^yelp\./
    }
  },
  {
    $lookup: {
      from: "shards",
      localField: "shard",
      foreignField: "_id",
      as: "shardInfo"
    }
  },
  {
    $project: {
      _id: 0,
      namespace: "$collectionInfo.id",
      shard: 1,
      shardHost: { $arrayElemAt: ["$shardInfo.host", 0] },
      min: 1,
      max: 1
    }
  },
  { $sort: { namespace: 1, shard: 1 } },
  { $limit: 20 }
]);
```

Vysvětlení: Dotaz pracuje přímo v konfigurační databázi. Propojuje informace o chunkech, kolekcích a shardech. Výsledkem jsou rozsahy chunků pro kolekce databáze yelp a informace, na kterém shardu jsou umístěny.

5. Kategorie 4 - Nested / embedded dokumenty

Dotaz 19: Počet podniků v jednotlivých kategoriích

Úloha: počet podniků v jednotlivých kategoriích

```

db.businesses.aggregate([
{
  $project: {
    category_list: {
      $cond: [
        { $isArray: "$categories" },
        "$categories",
        {
          $cond: [
            { $or: [{ $eq: ["$categories", null] }, { $eq: ["$categories", ""] } ] },
            [],
            { $split: ["$categories", "," ] }
          ]
        }
      ]
    }
  }
},
{ $unwind: "$category_list" },
{ $set: { category: { $trim: { input: "$category_list" } } } },
{ $match: { category: { $nin: [null, ""] } } },
{
  $group: {
    _id: "$category",
    business_count: { $sum: 1 }
  }
},
{
  $project: {
    _id: 0,
    category: "$_id",
    business_count: 1
  }
},
{ $sort: { business_count: -1 } },
{ $limit: 30 }
], { allowDiskUse: true });

```

Vysvětlení: Dotaz rozbaluje kategorii podniků a počítá, kolik podniků spadá do každé kategorie. Je ošetřena varianta, kdy jsou kategorie uloženy jako pole, i varianta, kdy jsou uloženy jako textový řetězec oddělený čárkami.

Dotaz 20: Nejlépe hodnocený podnik v každé kategorii

Úloha: nejlépe hodnocený podnik v každé kategorii

```

db.businesses.aggregate([
{
  $project: {
    name: 1,
    stars: 1,
    review_count: 1,
    category_list: {

```

```

$cond: [
  { $isArray: "$categories" },
  "$categories",
  {
    $cond: [
      { $or: [{ $eq: ["$categories", null] }, { $eq: ["$categories", ""] } ] },
      [],
      { $split: ["$categories", ","] }
    ]
  }
]
}
}
},
{ $unwind: "$category_list" },
{ $set: { category: { $trim: { input: "$category_list" } } } },
{ $match: { category: { $nin: [null, ""] } } },
{
  $sort: {
    category: 1,
    stars: -1,
    review_count: -1
  }
},
{
  $group: {
    _id: "$category",
    best_business_name: { $first: "$name" },
    best_stars: { $first: "$stars" },
    review_count: { $first: "$review_count" }
  }
},
{
  $project: {
    _id: 0,
    category: "$_id",
    best_business_name: 1,
    best_stars: 1,
    review_count: 1
  }
},
{ $sort: { best_stars: -1, review_count: -1 } },
{ $limit: 30 }
], { allowDiskUse: true });

```

Vysvětlení: Po rozbalení kategorií se podniky seřadí podle kategorie, hodnocení a počtu recenzí. Následné \$group s operátorem \$first vybere nejlepší podnik v každé kategorii. Dotaz vrací jiné informace než prostý počet kategorií, protože identifikuje konkrétní nejlepší podniky.

Dotaz 21: Uživatelé s nejvyšším počtem přátel

Úloha: uživatelé s nejvyšším počtem přátel

```

db.users.aggregate([
{
  $project: {
    name: 1,
    user_id: 1,
    friend_count: {
      $cond: [
        { $isArray: "$friends" },
        { $size: "$friends" },
        {
          $cond: [
            { $or: [{ $eq: ["$friends", null] }, { $eq: ["$friends", ""] } ] },
            0,
            { $size: { $split: ["$friends", ","] } }
          ]
        }
      ]
    }
  }
},
{ $sort: { friend_count: -1 } },
{ $limit: 10 }
], { allowDiskUse: true });

```

Vysvětlení: Dotaz počítá počet přátel uživatele. Je ošetřeno, zda je friends uložen jako pole nebo jako textový seznam. Výsledek ukazuje uživatele s nejrozsáhlejší sítí vazeb v datasetu.

Dotaz 22: Podniky s otevírací dobou alespoň pro pět dní v týdnu

Úloha: podniky s otevírací dobou alespoň pro pět dní v týdnu

```

db.businesses.aggregate([
{
  $project: {
    name: 1,
    city: 1,
    state: 1,
    days_open: {
      $size: {
        $objectToArray: {
          $ifNull: ["$hours", {}]
        }
      }
    }
  }
},
{ $match: { days_open: { $gte: 5 } } },
{ $sort: { days_open: -1, name: 1 } },
{ $limit: 20 }
], { allowDiskUse: true });

```

Vysvětlení: Pole hours je v dokumentu podniku uloženo jako vnořený objekt. Operátor \$objectToArray převede objekt na pole dvojic klíč-hodnota, aby bylo možné spočítat počet dní s definovanou otevírací dobou.

Dotaz 23: Počet podniků podle hodnoty embedded atributu WiFi

Úloha: počet podniků podle hodnoty embedded atributu WiFi

```
db.businesses.aggregate([
  {
    $match: {
      "attributes.WiFi": { $exists: true, $nin: [null, ""] }
    }
  },
  {
    $project: {
      wifi: "$attributes.WiFi"
    }
  },
  {
    $group: {
      _id: "$wifi",
      business_count: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      wifi: "$_id",
      business_count: 1
    }
  },
  { $sort: { business_count: -1 } }
], { allowDiskUse: true });
```

Vysvětlení: Dotaz pracuje přímo s vnořeným atributem attributes.WiFi. Výsledkem je souhrn, kolik podniků má jednotlivé hodnoty WiFi atributu. Jde o typický příklad dotazování do embedded dokumentu.

Dotaz 24: Počet elitních uživatelů podle jednotlivých let

Úloha: počet elitních uživatelů podle jednotlivých let

```
db.users.aggregate([
  {
    $project: {
      elite_years: {
        $cond: [
          { $isArray: "$elite" },
          "$elite",
          {
            $cond: [
              { $or: [{ $eq: ["$elite", null] }, { $eq: ["$elite", ""] } ] },
              [],
              { $split: ["$elite", ","] }
            ]
          }
        ]
      }
    }
  }
])
```

```

    ]
  }
]
}
},
{ $unwind: "$elite_years" },
{ $set: { elite_year: { $trim: { input: { $toString: "$elite_years" } } } } },
{ $match: { elite_year: { $nin: [null, ""] } } },
{
  $group: {
    _id: "$elite_year",
    elite_user_count: { $sum: 1 }
  }
},
{
  $project: {
    _id: 0,
    elite_year: "$_id",
    elite_user_count: 1
  }
},
{ $sort: { elite_year: 1 } }
], { allowDiskUse: true });

```

Vysvětlení: Pole elite obsahuje roky, ve kterých byl uživatel označen jako elitní. Dotaz rozbílí jednotlivé roky a spočítá počet výskytů pro každý rok. Je ošetřena varianta pole i textového seznamu.

6. Kategorie 5 - Indexy

Dotaz 25: Vytvoření compound indexu nad reviews podle business_id a date

Úloha: vytvoření compound indexu nad reviews podle business_id a date

```

db.reviews.createIndex(
  { business_id: 1, date: -1 },
  { name: "idx_reviews_business_date" }
);

db.reviews.getIndexes().forEach(function(idx) {
  printjson(idx);
});

```

Vysvětlení: Compound index nad business_id a date je vhodný pro dotazy, které hledají recenze konkrétního podniku a řadí je podle data. Ověřovací getIndexes vrací seznam indexů v kolekci reviews.

Dotaz 26: Ověření využití indexu nad reviews na existujícím business_id

Úloha: ověření využití indexu nad reviews na existujícím business_id

```
const sampleReviewForIndex = db.reviews.findOne(
  { business_id: { $exists: true, $nin: [null, ""] } },
  { business_id: 1 }
);
```

```
db.reviews.find(
  { business_id: sampleReviewForIndex.business_id },
  { _id: 0, business_id: 1, stars: 1, date: 1, text: 1 }
).sort({ date: -1 }).limit(10);
```

```
db.reviews.find(
  { business_id: sampleReviewForIndex.business_id }
).sort({ date: -1 }).explain("executionStats");
```

Vysvětlení: Místo napevno zadaného business_id se nejprve vybere existující hodnota z aktuálního datasetu. Díky tomu dotaz funguje i u zmenšeného vzorku dat. První find vrací recenze, druhý explain ukazuje plán vykonání a umožňuje ověřit použití indexu idx_reviews_business_date.

Dotaz 27: Vytvoření compound indexu nad users podle yelping_since a review_count

Úloha: vytvoření compound indexu nad users podle yelping_since a review_count

```
db.users.createIndex(
  { yelping_since: 1, review_count: -1 },
  { name: "idx_users_since_review_count" }
);

db.users.getIndexes().forEach(function(idx) {
  printjson(idx);
});
```

Vysvětlení: Index je určen pro dotazy, které filtrují uživatele podle data registrace a zároveň pracují s počtem recenzí. Výpis getIndexes dokládá, že index byl vytvořen.

Dotaz 28: Ověření využití indexu nad users

Úloha: ověření využití indexu nad users

```
db.users.find(
  {
    yelping_since: { $gte: "2015-01-01" },
    review_count: { $gte: 50 }
  },
  {
    _id: 0,
    user_id: 1,
    name: 1,
    yelping_since: 1,
    review_count: 1
  }
).sort({ review_count: -1 }).limit(10);
```

```

db.users.find(
{
  yelping_since: { $gte: "2015-01-01" },
  review_count: { $gte: 50 }
}
).sort({ review_count: -1 })
.hint("idx_users_since_review_count")
.explain("executionStats");

```

Vysvětlení: První část dotazu vrací konkrétní uživatele splňující filtr. Druhá část využívá explain a hint, aby bylo zřetelně doloženo použití vytvořeného indexu. Dotaz vrací jiný typ dat než předchozí indexový dotaz nad recenzemi.

Dotaz 29: Vytvoření compound indexu nad businesses podle lokality a hodnocení

Úloha: vytvoření compound indexu nad businesses podle lokality a hodnocení

```

db.businesses.createIndex(
{ city: 1, state: 1, is_open: 1, stars: -1 },
{ name: "idx_businesses_city_state_open_stars" }
);

db.businesses.getIndexes().forEach(function(idx) {
  printjson(idx);
});

```

Vysvětlení: Tento index podporuje dotazy, které filtrují podniky podle města, státu a otevřenosti a následně je řadí podle hodnocení. Jde o typický uživatelský scénář hledání podniků v konkrétní lokalitě.

Dotaz 30: Ověření využití indexu nad businesses na lokálním dotazu

Úloha: ověření využití indexu nad businesses na lokálním dotazu

```

db.businesses.find(
{
  city: "Philadelphia",
  state: "PA",
  is_open: 1
},
{
  _id: 0,
  name: 1,
  stars: 1,
  review_count: 1,
  city: 1,
  state: 1
}
).sort({ stars: -1 }).limit(10);

db.businesses.find(
{
  city: "Philadelphia",

```

```

    state: "PA",
    is_open: 1
  }
).sort({ stars: -1 })
.hint("idx_businesses_city_state_open_stars")
.explain("executionStats");

```

Vysvětlení: První část vrací otevřené podniky ve městě Philadelphia ve státě PA, seřazené podle hodnocení. Druhá část ukazuje plán vykonání dotazu a pomocí hint vynucuje použití indexu idx_businesses_city_state_open_stars.

7. Mimo 30 hlavních dotazů - simulace výpadku uzlu

7.1 Výchozí ověření replica setu

```

docker compose exec shard1a mongosh \
"mongodb://127.0.0.1:27018/?directConnection=true"

```

```

rs.status().members.forEach(function(m) {
  printjson({
    _id: m._id,
    uzel: m.name,
    stav: m.stateStr,
    zdravi: m.health
  });
});

```

Nejprve se ověří aktuální stav členů replica setu. Výpis ukáže, který uzel je PRIMARY, které uzly jsou SECONDARY a zda jsou jednotlivé členy zdravé.

7.2 Zastavení sekundárního uzlu

```

docker compose stop shard1b

```

```

docker compose exec shard1a mongosh \
"mongodb://127.0.0.1:27018/?directConnection=true"

```

```

rs.status().members.forEach(function(m) {
  printjson({
    _id: m._id,
    uzel: m.name,
    stav: m.stateStr,
    zdravi: m.health
  });
});

```

Po zastavení sekundárního uzlu zůstává shard díky replica setu dostupný. Primární uzel nadále obsluhuje zápis i čtení podle konfigurace clusteru. Ve výpisu rs.status() je u zastaveného uzlu viditelný problém se stavem nebo zdravím uzlu.

7.3 Návrat uzlu zpět

`docker compose start shard1b`

`docker compose exec shard1a mongosh \`
`"mongodb://127.0.0.1:27018/?directConnection=true"`

```
rs.status().members.forEach(function(m) {  
  printjson({  
    _id: m._id,  
    uzel: m.name,  
    stav: m.stateStr,  
    zdravi: m.health  
  });  
});
```